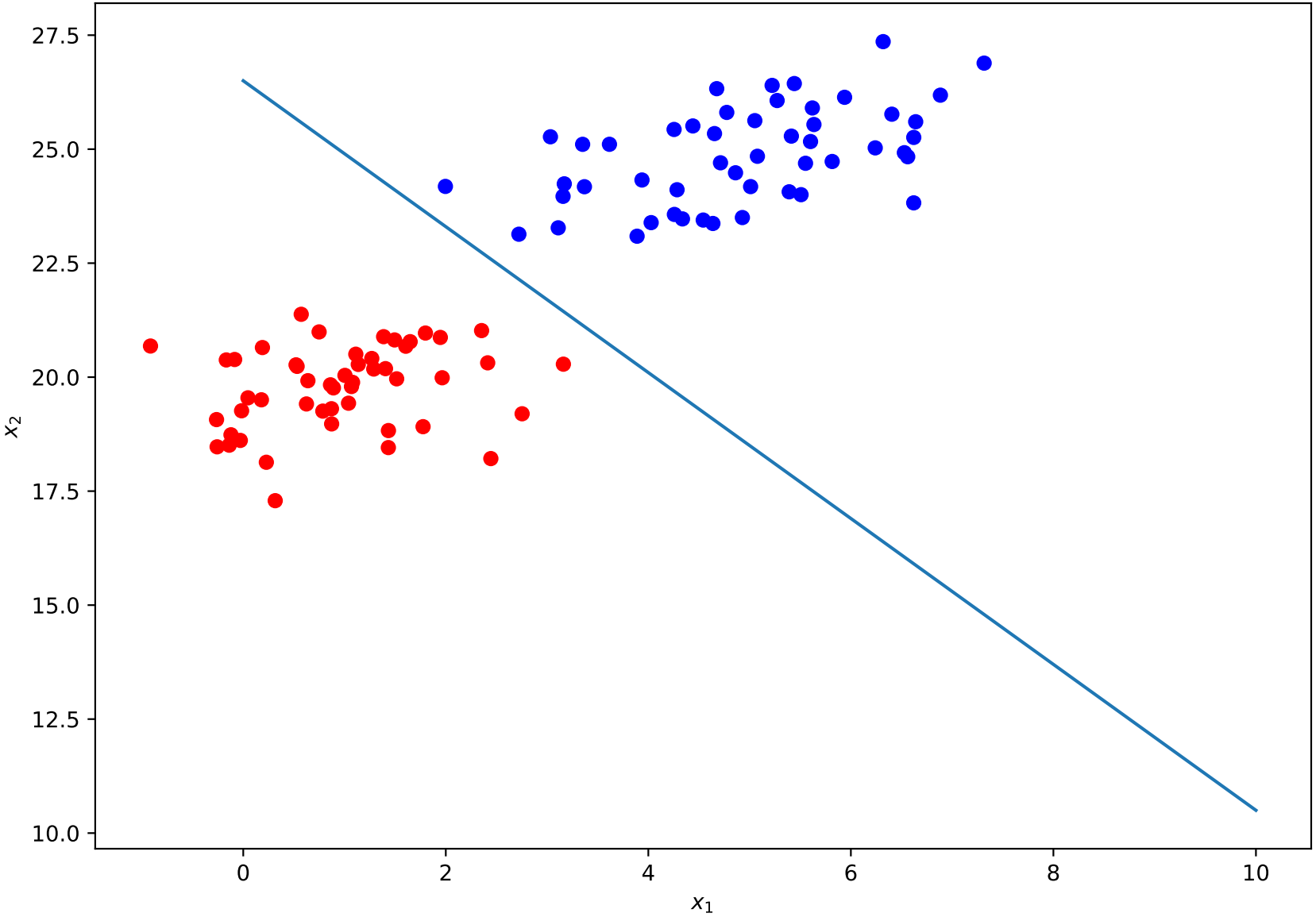
Support Vector Machines (SVM)

Support vector machine

- A classification method. We focus on the case with two classes.
- Maximal margin classifier: Separate classes completely with linear boundaries
- Support vector classifier: Complete separation is not necessary
- Support vector machines: Also allow for non-linear boundaries

A separating line (hyperplane)



Hyperplanes

- Example: $p = 2$ (2 predictors). A hyperplane is defined by

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

$$\Longleftrightarrow$$

$$X_2 = -\frac{\beta_0}{\beta_2} - \frac{\beta_1}{\beta_2} X_1 \qquad \square$$

- If $p = 3$ an hyperplane is given by

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 = 0$$

  and is a plane (2-dimensional object)

- In general, a hyperplane in a p-dimensional space is a $p-1$-dimensional object which separates the p-dimensional space into two pieces.

Classification by separating hyperplanes

- We have a data matrix with n training observations and p variables (predictors)

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

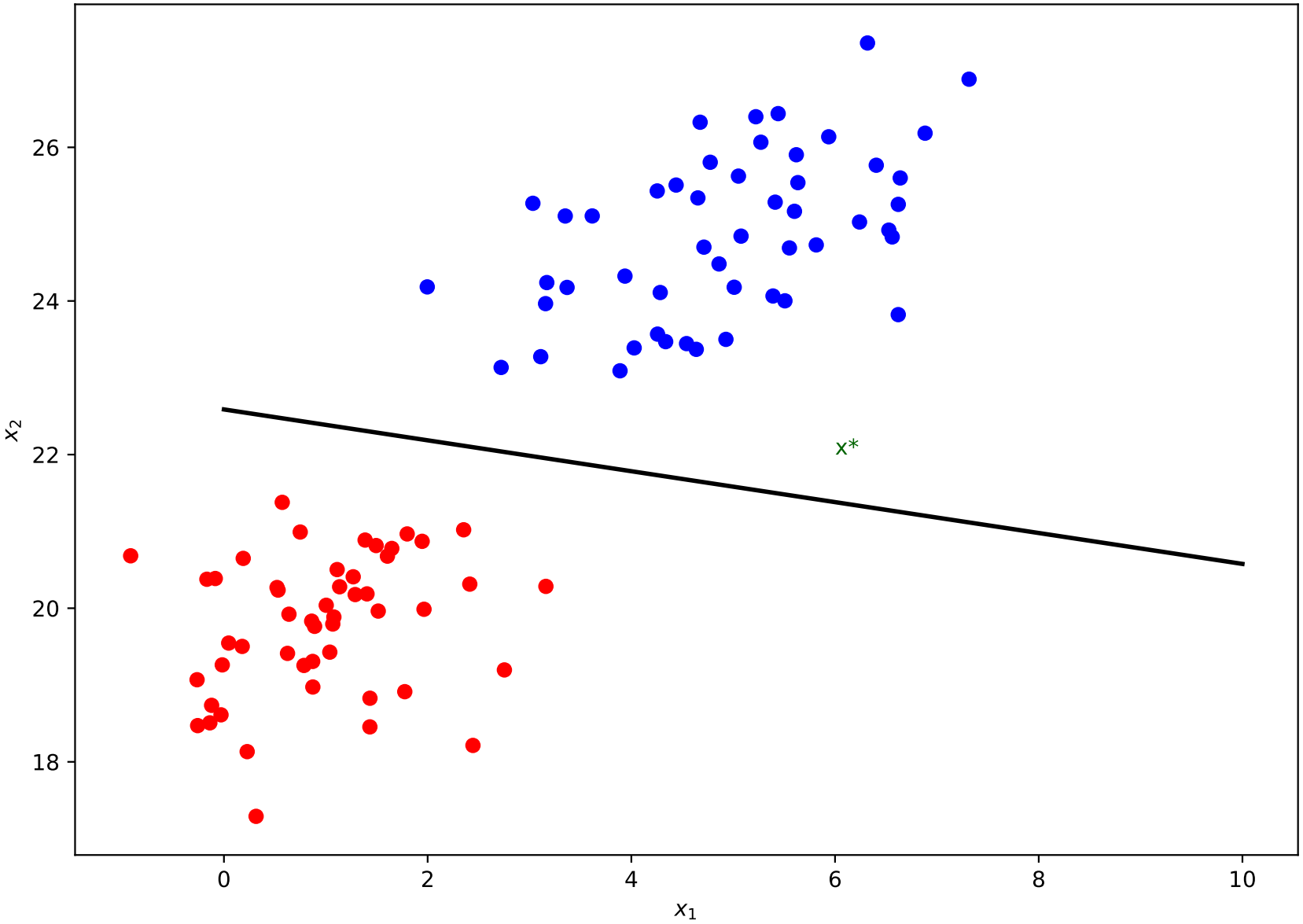
- Each observation/individual i, is represented by a row

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{bmatrix}, \quad i = 1, 2, \ldots, n$$

- Each observation falls into one of two classes , represented by $y_i \in \{-1, 1\}$

Classification by separating hyperplanes

We would like to classify a test observation x^* (p-vector)



Classification by separating hyperplanes

In general predict

- $y_i = 1$ if

$$f(x^*) = \beta_0 + \beta_1 x^*_{i1} + \beta_2 x^*_{i2} + ... + \beta_p x^*_{ip} > 0$$

- and $y_i = -1$ if

$$f(x^*) < 0$$

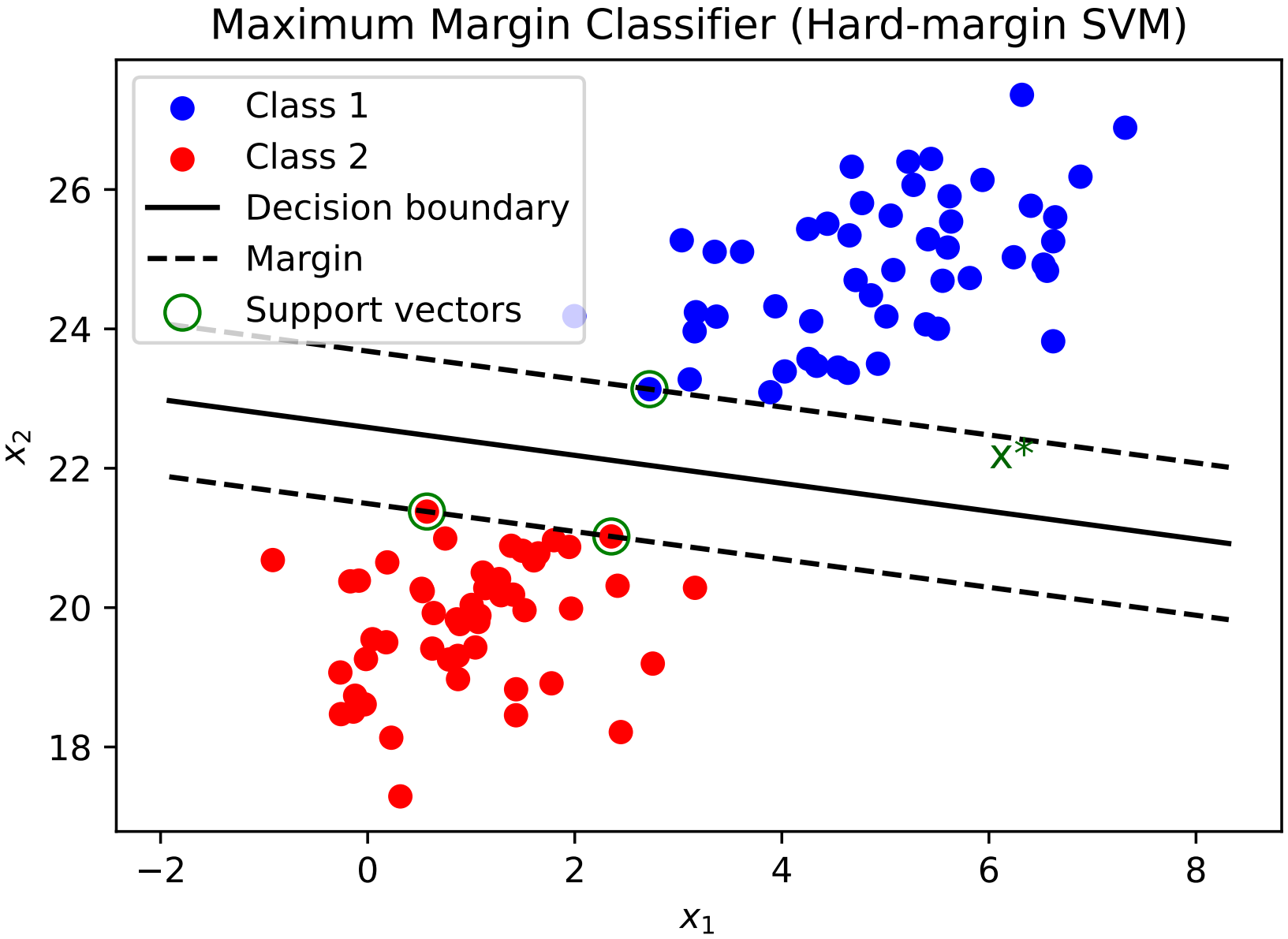
- This implies that a separating hyperplane has the property that

$$y_i f(x^*) > 0$$

  for all $i = 1, 2, ..., n$.

- If the magnitude (absolute value) of $f(x^*)$ is large we can be confident about our class assignment.

The maximal margin classifier



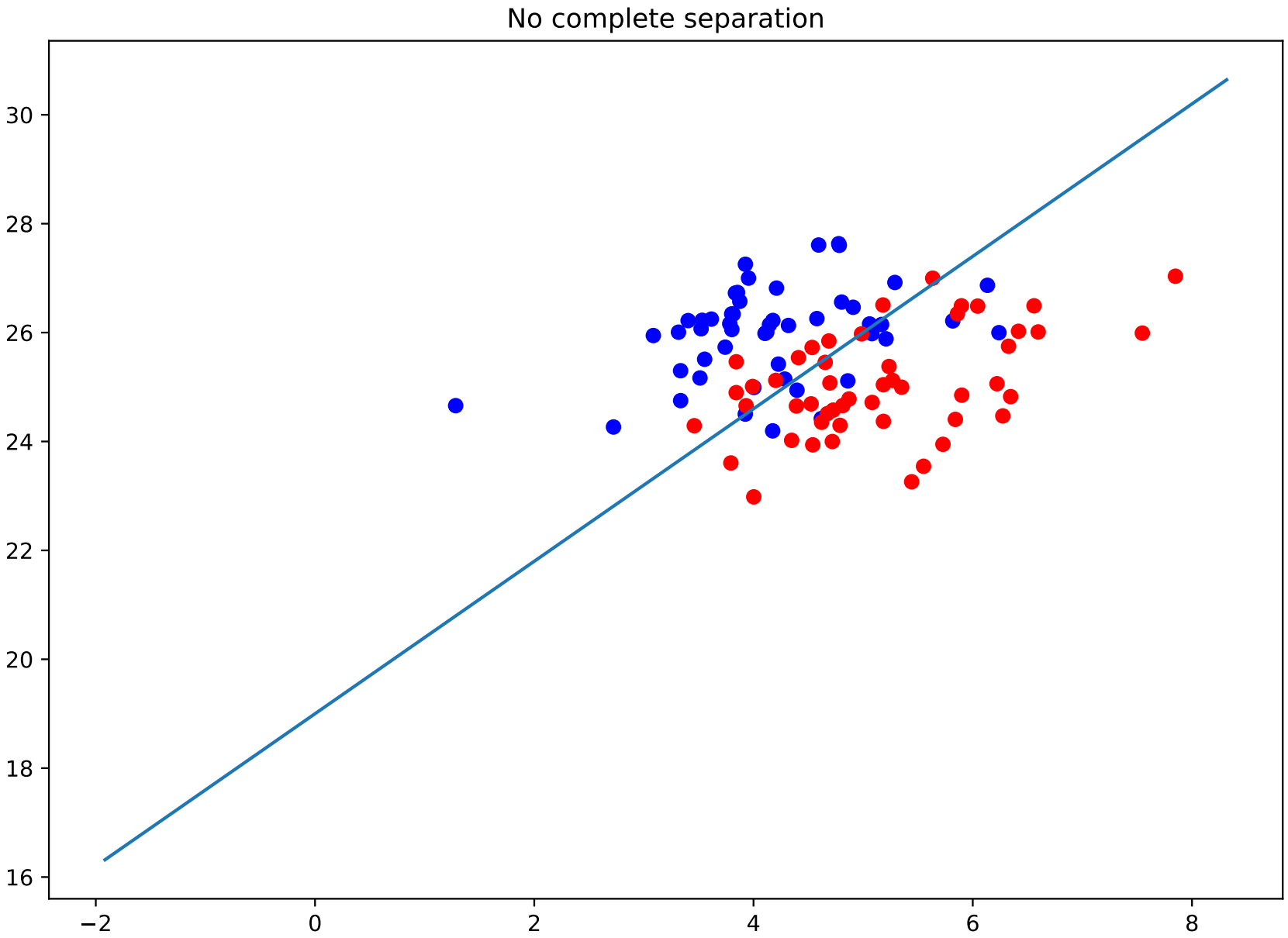
- The maximal margin classifier is the separating hyperplane

The maximal margin classifier

- $$\begin{cases} \underset{\beta_0,\beta_1,\ldots,\beta_p}{\text{maximize}}\, M \\ \text{subject to} \\ \sum_{j=1}^{p} \beta_j^2 = 1 \\ y_i(\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \ldots + \beta_p x_{ip}^*) \geq M \end{cases}$$

- The restriction $\sum_{j=1}^{p} \beta_j^2 = 1$ is necessary to make $y_i f(x^*)$ the distance between the points and the hyperplane.

No separating hyperplane



Support vector classifier (soft margin classifier)

- We like to allow for some misclassified observations; allow them to be on the wrong side of the margin (red and blue line) or even on the wrong side of the separating hyperplane.
- $$\begin{cases} \underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n}{\text{maximize}} \quad M \\ \text{subject to} \\ \sum_{j=1}^{p} \beta_j^2 = 1 \\ y_i(\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \ldots + \beta_p x_{ip}^*) \geq M(1-\epsilon_i) \\ \epsilon_i \geq 0, \ \sum_{i=1}^{n} \epsilon_i \leq K \end{cases}$$
- K is a tuning parameter that can be found by...
- Classification is done as it was for the maximal margin classifier.

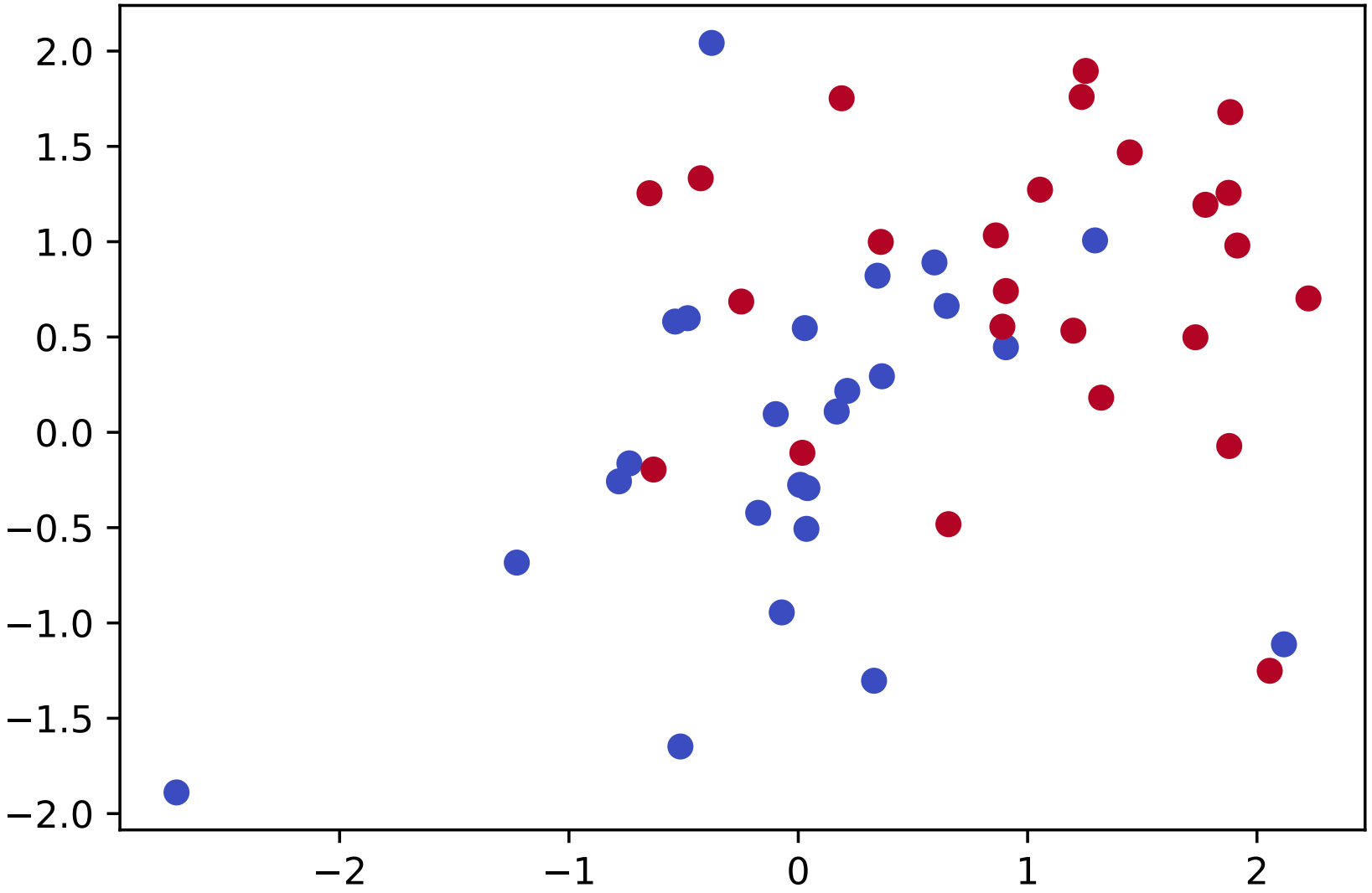
Support vector classifier

- Only observations violating the margin, or on the margin, affect the solution
- The *slack variables* $\epsilon_1, ..., \epsilon_n$ allows the observations to be on the wrong side of the margins or hyperplane.
- They tell us where the observations are located.
- $\epsilon_i = 0$ means that the ith observation is on the correct side of the margin.
- $\epsilon_i > 0$ means that the ith observation is on the wrong side of the margin.

The constant K regulates how much the margin can be violated

- If $K = 0$ we do not allow for any violations and we have the maximal margin classifier.
- If $K > 0$, at most K observations can be on the wrong side of the hyperplane.
- A large K means that we allow more violations; we have a wide margin.
- K is a tuning parameter that can be found by cross-validation.

A simulated data set (page 387 in ISLP)



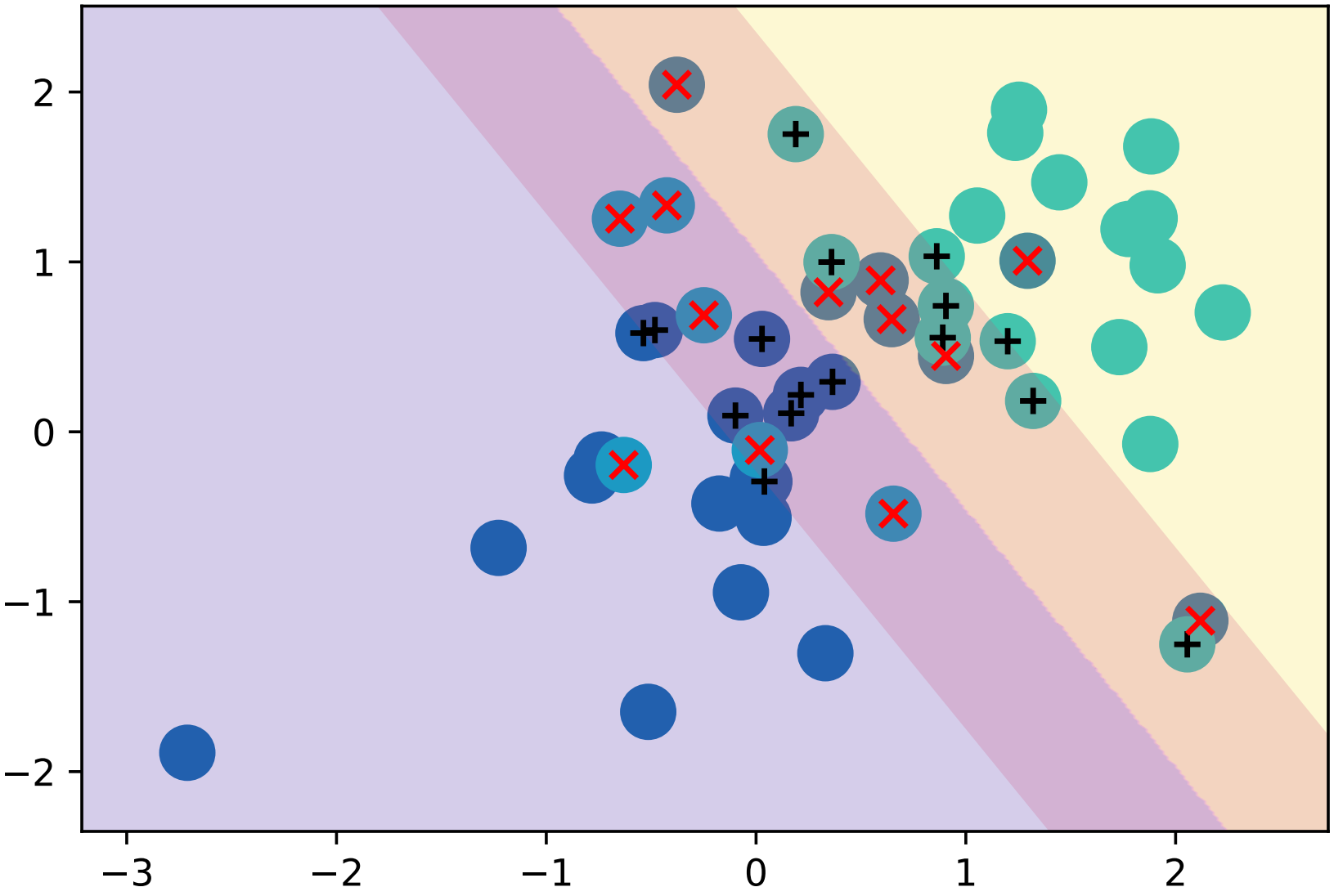
A simulated data set

A large K corresponds to a small C in the function `SVC`.

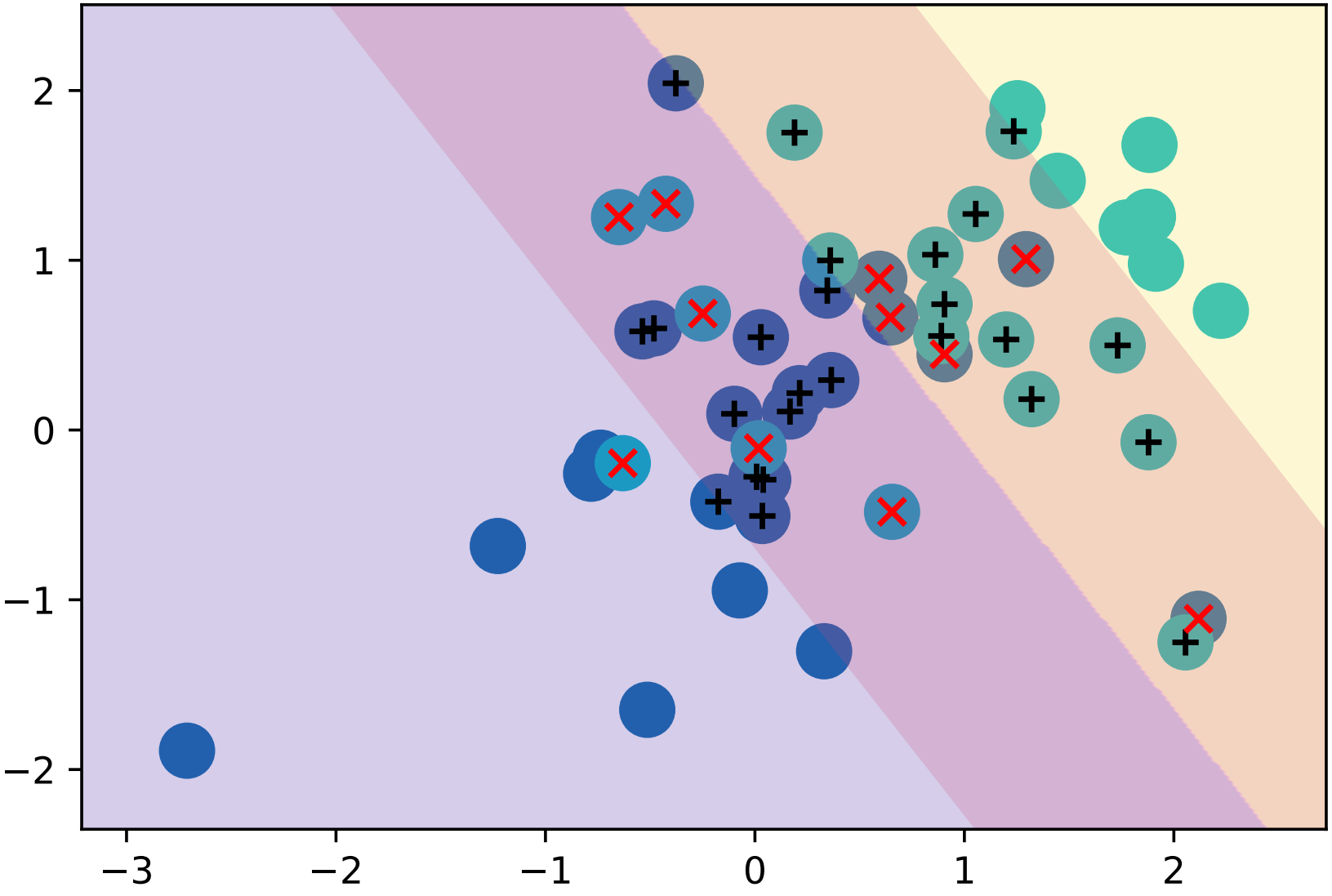
```
from sklearn.svm import SVC
from ISLP.svm import plot as plot_svm
from sklearn.metrics import RocCurveDisplay
from ISLP import confusion_table
roc_curve = RocCurveDisplay.from_estimator
svm_linear = SVC(C=10, kernel='linear')
svm_linear.fit(X, y)
fig , ax = plt.subplots(figsize =(8 ,8))
plot_svm(X,y,svm_linear,ax=ax)
```

A simulated data set

A large K corresponds to a small C in the function `SVC`.



A larger K (smaller C)



Finding the optimal K (and C)

```
import sklearn.model_selection as skm
kfold = skm.KFold(5,random_state=0,shuffle=True)
grid = skm.GridSearchCV(svm_linear,
    {'C':[0.001 ,0.01 ,0.1 ,1 ,5 ,10 ,100]},
refit=True ,cv=kfold, scoring="accuracy")
grid.fit(X, y)
print(grid.best_params_)
print(grid.cv_results_[('mean_test_score')])
```

```
{'C': 1}
[0.46 0.46 0.72 0.74 0.74 0.74 0.74]
```

Predictions on 20 test observations

```
best_ = grid.best_estimator_
y_test_hat = best_.predict(X_test)
confusion_table(y_test_hat , y_test)
```

Truth Predicted	-1	1
-1	8	4
1	2	6

Support vector machines

Consider the support vector classifier problem again

$$\begin{cases} \underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n}{\text{maximize}} \quad M \\ \text{subject to} \\ \sum_{j=1}^{p} \beta_j^2 = 1 \\ y_i \underbrace{(\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \ldots + \beta_p x_{ip}^*)}_{f(x^*)} \geq M(1-\epsilon_i),\ i = 1, \ldots, n \\ \epsilon_i \geq 0,\ \sum_{i=1}^{n} \epsilon_i \leq C \end{cases}$$

The decision boundary, $f(x^*)$, must not necessarily be linear in x^*.

Support vector machines

- We let

$$f(x^*) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(x^*, x_i),$$

  where K is a *kernel*.

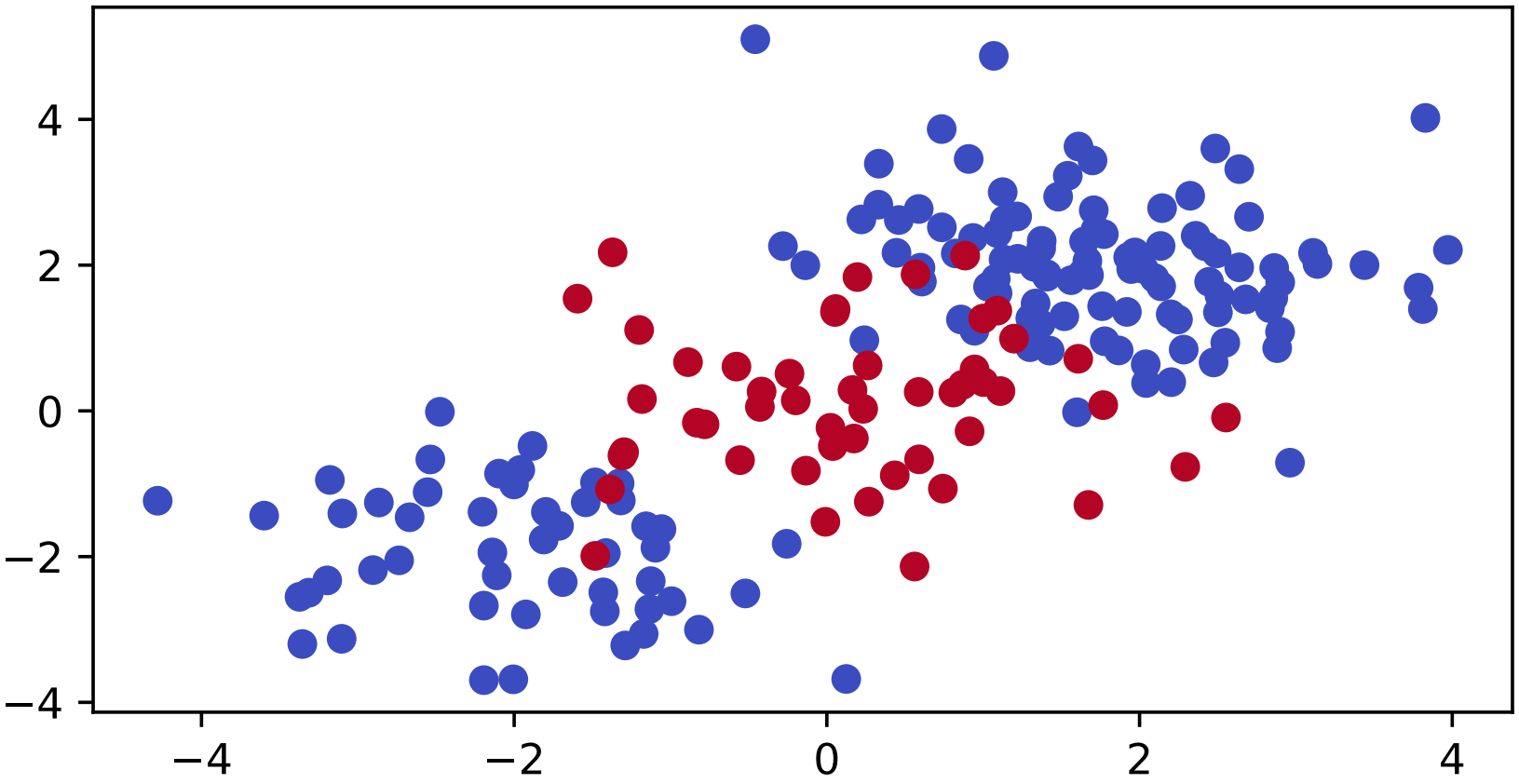
- Two commonly used kernels are the *polynomial kernel* of degree d

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^{p} x_{ij} x_{i'j}\right)^d$$

- and the *radial kernel*

$$K(x_i, x_{i'}) = \exp\left(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2\right)$$

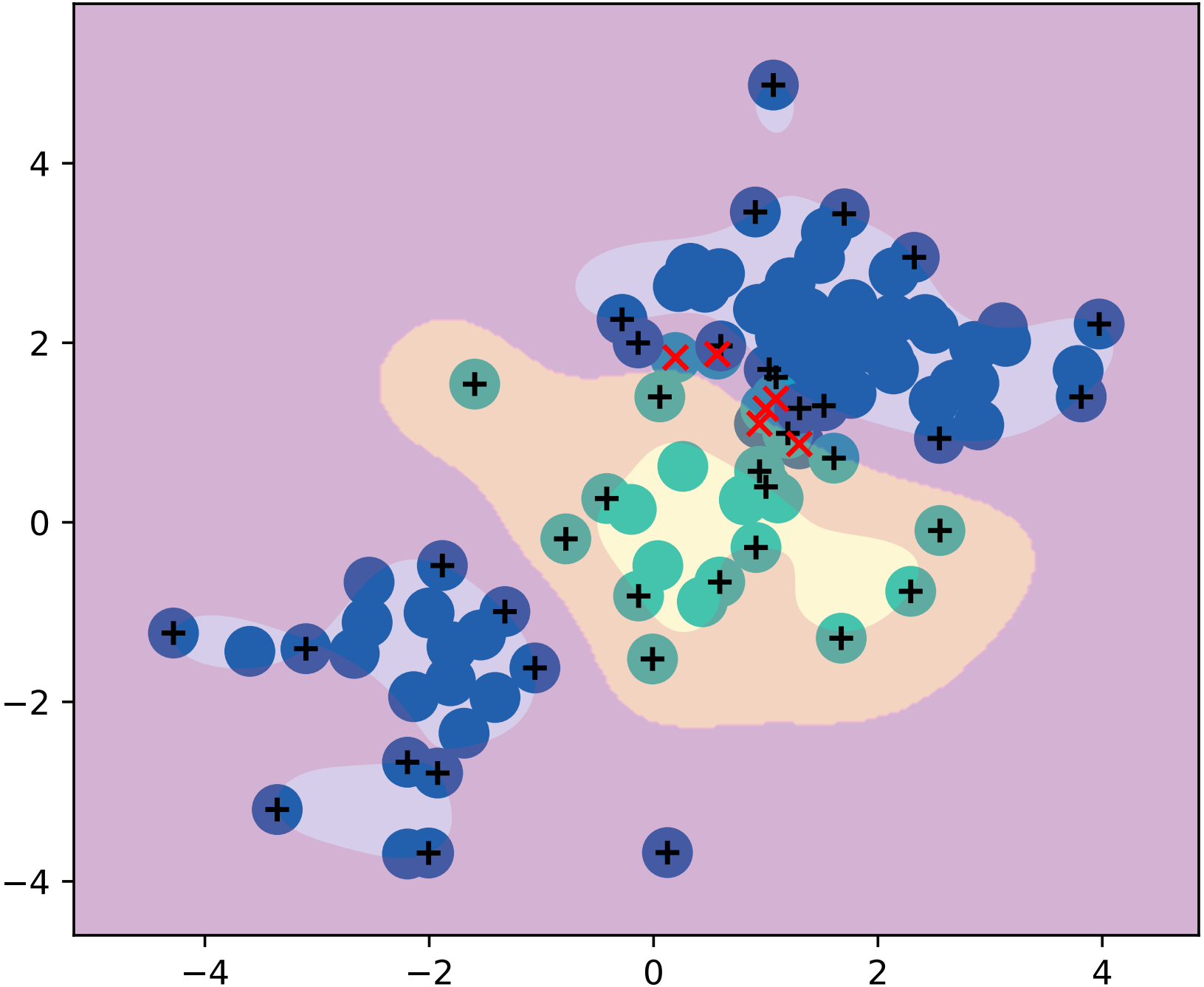
An example



An example

```
# Radial kernel
(X_train,X_test,y_train,y_test) = skm.train_test_split(
  X,y,test_size=0.5, random_state =0)
svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train , y_train);
```

An example



Confusion matrix for test data

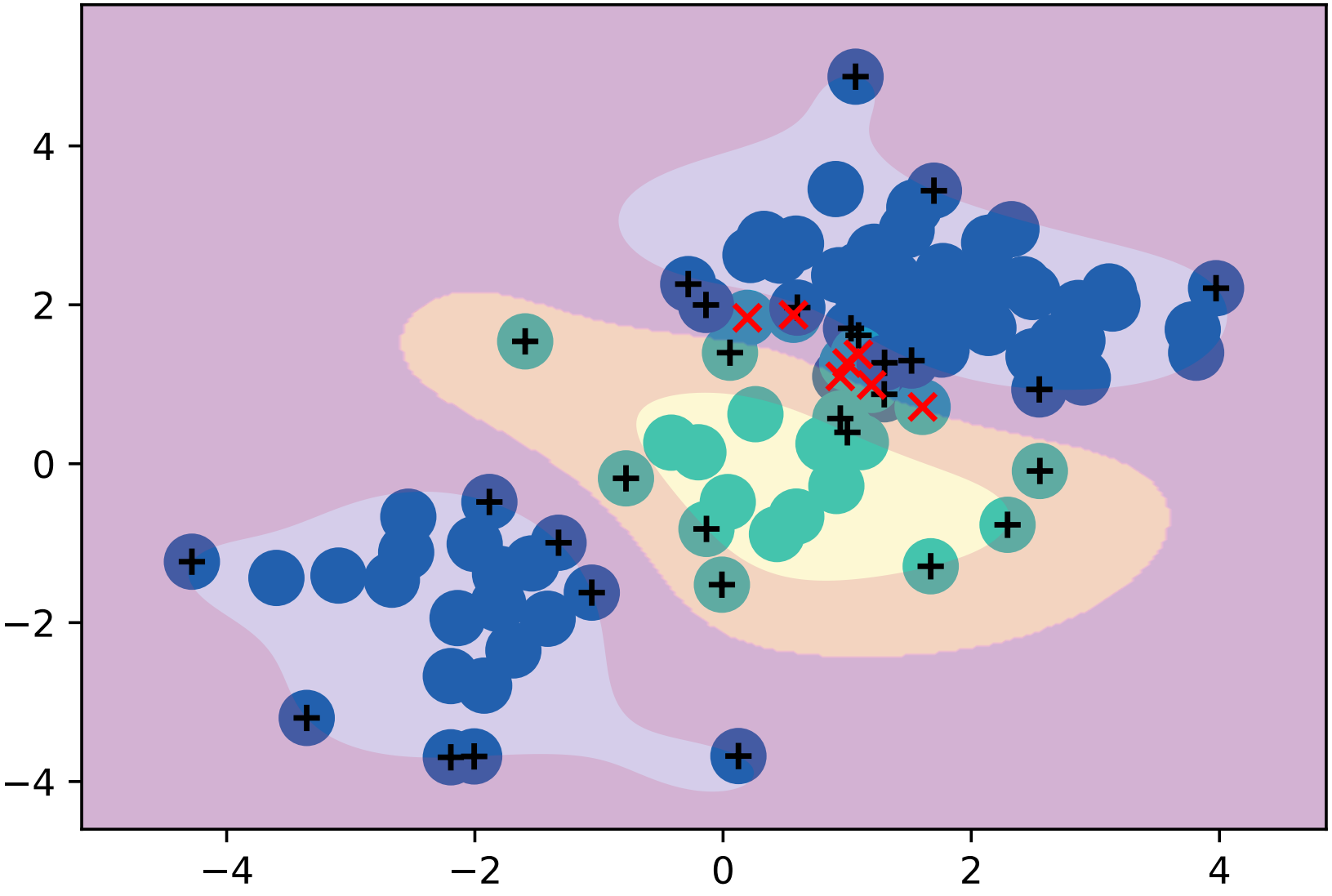
Truth Predicted	1	2
1	67	6
2	8	19

Optimal C and gamma

```
kfold = skm.KFold(5,
random_state=0,
shuffle=True)
grid = skm.GridSearchCV(svm_rbf ,
{'C':[0.1 ,1 ,10 ,100 ,1000] ,
'gamma':[0.5,1,2 ,3,4]},
refit=True ,
cv=kfold ,
scoring='accuracy');
grid.fit(X_train , y_train)
print(grid.best_params_)
```

```
{'C': 1, 'gamma': 0.5}
```

Training fit for the best model



Confusion matrix for test data

```
best_svm = grid.best_estimator_
y_hat_test = best_svm.predict(X_test)
confusion_table(y_hat_test , y_test)
```

Truth Predicted	1	2
1	69	6
2	6	19